

COMICCRAFT

Software Development Life Cycle (SDLC) — Presentation Script

Topic: AI-powered 5-panel comic generation platform

Total Duration: Approximately 9.5 minutes

Technology Background

ComicCraft combines a web application architecture with Generative AI. The frontend collects story inputs and displays generated panels; FastAPI coordinates application logic; Gemini is used for narrative planning; Stable Diffusion/Diffusers is used for image generation; and FPDF compiles the final comic into a downloadable PDF.

Core SDLC Flow

Ideation → Requirements → Design → Planning → Development → Testing → Documentation → Demonstration & Delivery

1. Introduction, Purpose & SDLC Overview (~1:15)

Good morning, everyone! Today, I'm presenting the Software Development Life Cycle for ComicCraft—an AI-powered platform that transforms story prompts into fully visual 5-panel comic books.

The purpose of our SDLC is simple: turning non-deterministic Generative AI tools into a reliable, production-ready application. Building with AI involves unpredictable text outputs, API latency, and heavy image rendering. Without a structured lifecycle, software quickly becomes unstable and difficult to maintain.

Think of this SDLC like building a home. You don't lay bricks without architectural blueprints. In our project, the SDLC serves as our master plan—guaranteeing that every component, from frontend forms to backend AI calls, works in perfect harmony across eight structured phases.

2. Brainstorming & Ideation Phase (~1:00)

Phase 1 was Brainstorming & Ideation. We set out to solve a core challenge: How can non-artists bring visual stories to life? Creating comics usually requires both creative writing and detailed illustration skills.

During this phase, we evaluated Generative AI models for speed, cost, and output quality. We selected Google Gemini models for storytelling and narrative planning, and Stable Diffusion via Hugging Face Diffusers for panel-by-panel artwork.

We defined our core product vision: generating a sequential 5-panel story arc with customizable art styles—like Manga or Cyberpunk—and providing downloadable PDF exports.

3. Requirement Analysis Phase (~1:10)

Phase 2 was Requirement Analysis, where we translated ideas into technical specifications.

For Functional Requirements, the application must accept user inputs for prompts, character names, and visual styles; parse story ideas into 5 visual panels; generate corresponding images; and offer a downloadable PDF.

For Non-Functional Requirements, we prioritized thread performance and clean API error handling so external timeouts never crash the server.

To meet these needs, we selected our tech stack: FastAPI for an asynchronous Python backend, Jinja2 for lightweight frontend views, and FPDF for document compilation.

4. Project Design Phase (~1:10)

Phase 3 was Project Design. We architected ComicCraft using a classic 3-Tier Model:

Presentation Layer: Jinja2 templates handling user inputs and comic display.

Application Layer: FastAPI backend routes orchestrating business logic and API requests.

Integration Layer: External AI endpoints and local file storage.

We established a clear data pipeline: user inputs trigger a call to Gemini Flash, which returns a structured 5-panel JSON outline. The backend loops through this outline, requests images from Diffusers, and saves them to static storage. Clean REST endpoints like `/generate` and `/download-pdf` keep the system modular and decoupled.

5. Project Planning Phase (~1:00)

Phase 4 was Project Planning. Before writing code, we established strict project organization and environment security.

We defined a clean directory hierarchy separating backend routes (`app/`), Jinja views (`templates/`), and output assets (`static/`).

Security was a top priority: all third-party API keys were isolated in protected `.env` files using `python-dotenv`, preventing secrets from being hardcoded. We locked dependencies in `requirements.txt` and divided execution into milestones—from base server setup to final PDF compilation.

6. Development Phase (~1:15)

Phase 5 was Development. Implementation began by setting up FastAPI routes and mounting static directories.

A key focus was prompt engineering. To ensure Gemini returned valid, machine-readable story structures, we crafted system instructions enforcing strict JSON output formats containing panel descriptions, dialogues, and captions.

On the visual side, we appended art style modifiers to prompts before querying image generation models. Finally, we built the PDF assembly module using `FPDF`, writing custom layout logic to draw images, borders, and dialogue bubbles onto a downloadable canvas.

7. Testing Phase (~1:10)

Phase 6 was Testing. Testing AI applications requires isolating variable outputs.

We built a dedicated `GET /test-image` endpoint to test image generation and file permissions independently, avoiding unnecessary LLM token usage.

We implemented robust `try-except` blocks to catch potential JSON parsing errors from LLM responses, ensuring the server gracefully handles malformed data. We also tested boundary inputs and verified that text captions auto-wrap cleanly inside PDF panel margins without overflowing.

8. Documentation Phase (~1:00)

Phase 7 was Documentation. To make the codebase maintainable, we leveraged FastAPI's auto-generation features.

By using type hints and docstrings, FastAPI automatically builds interactive Swagger documentation at /docs. Developers can test endpoints and review JSON request schemas live in their browser.

We also authored a detailed README.md covering environment setup, API keys, and launch commands, alongside internal function docstrings to streamline developer onboarding.

9. Demonstration & Conclusion (~1:00)

Our final phase was Demonstration & Delivery.

In the live app, a user enters a story idea, picks a character and art style, and hits Generate. Within seconds, Gemini generates the script, Stable Diffusion renders the artwork, and the user views an interactive comic strip on screen. A single click on 'Download PDF' delivers a publication-ready document.

Following these eight structured SDLC phases allowed us to turn complex Generative AI tools into a simple, reliable web product. Thank you! I am happy to take any questions.